Parallel Image Processing and CNN Primitives with OpenMP

MTCS-103P — High Performance Computing Laboratory · Mini Project

Institution: Sri Sathya Sai Institute of Higher Learning
Topic: Image Processing (2) — CNN padding & pooling, and basic image processing
Implementation: C99 + OpenMP, every kernel written from scratch
Repository: https://github.com/buvann-11/Parallel-Image-Processing-OpenMP

Individual contributions

Registration no.	Name	Contribution
26010303001	B. Sai Buvann	**Part 1 — CNN primitives:** padding, convolution, pooling; OpenMP parallelisation strategy; C correctness test suite
26010303004	M. Sai Praneeth	**Part 2 — image processing:** greyscale, normalisation, resize; benchmark harness and scaling study; GUI; report

Work done by B. Sai Buvann (26010303001)

- **Padding** (`src/padding.c`) — `pad2d_serial/omp` on NCHW tensors with `zeros`, `reflect`, `replicate` and `circular` modes, including pads wider than the image axis. 3.42× on 2 threads.
- **Convolution and filtering** (`src/convolution.c`) — batched multi-channel `conv2d_serial/omp` with stride, dilation and bias; depthwise `filter_image_serial/omp` with box, Gaussian, Sobel, Laplacian, sharpen and emboss kernels; vectorised inner loop. 1.95–2.12× on 2 threads.
- **Pooling** (`src/pooling.c`) — max and average pooling with window, stride and padding; global average pooling. 2.33× on 2 threads.
- **OpenMP strategy** — row ownership for every kernel (no atomics, no critical sections, one barrier in the whole project), 64-byte-aligned buffers and cache-line-padded per-thread data to prevent false sharing, directive-free serial baselines.
- **C test suite** (`tests/test_correctness.c`) — 27 tests, 41,119 assertions, including serial = OpenMP, identical output at 1–4 threads and under all schedules. This suite exposed the compiler bug in §5.

Work done by M. Sai Praneeth (26010303004)

- **Greyscale** (`src/grayscale.c`) — BT.601, BT.709, mean, and gamma-correct luminance. 1.87× (memory-bound) and 1.99× (compute-bound) on 2 threads.

- **Normalisation** (`src/normalize.c`) — min–max, z-score and histogram equalisation as a parallel reduction followed by a parallel map; per-thread padded histograms. 2.14× (min–max) and 1.53× (equalisation).
- **Resize** (`src/resize.c`) — nearest, bilinear and bicubic, separable two-pass with anti-aliasing on downscale; fixed the `float32` sampling bug found by cross-validation. 1.57× (bicubic).
- **Benchmark harness** (`bench/benchmark.c`) — strong-scaling study with speedup, efficiency and Karp–Flatt metric; schedule and chunk-size experiments; all figures.
- **GUI** (`gui/app.py`) — Operations tab and one-click Test-suite tab.
- **Report** — writing, figures and screenshots.

---

1. Objective

Implement the operations underneath CNN-based image processing — padding, convolution and pooling (Part 1), and greyscale, normalisation and resize (Part 2) — parallelise each with OpenMP, and measure and explain how well each one scales. Every kernel exists twice: `<name>_serial()` with no OpenMP at all, and `<name>_omp()`; speedup is always measured against the directive-free serial version.

2. Implementation

Operation	Variants	Key design point
Padding	zeros, reflect, replicate, circular	Index folding, so any pad width works
Convolution	N×C×H×W, stride, dilation, bias; 7 image filters	Direct loop nest, output column innermost → auto-vectorised
Pooling	max, average, global average	Windows clamped to real input — padding never wins a max
Greyscale	BT.601, BT.709, mean, linear luminance	Luminance linearises sRGB before weighting
Normalisation	min–max, z-score, equalisation	Global reduction, then parallel map (LUT for equalisation)
Resize	nearest, bilinear, bicubic	Separable two-pass (4× less arithmetic for bicubic), anti-aliased

Images are read and written as PPM/PGM natively, and as PNG, JPEG, BMP or TGA through the public-domain stb headers; all formats give bit-identical results.


Input (PNG, 512x512)


Pad: reflect, 64 px


Conv: Gaussian 5x5

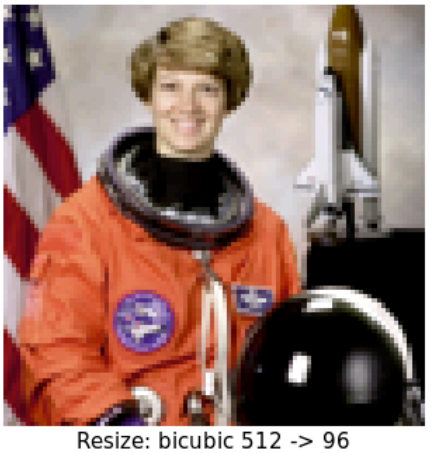

Conv: Sobel x


Conv: emboss


Max-pool 4x4, stride 4 (128x128)


Greyscale: linear luminance


Normalize: histogram equalisation


Resize: bicubic 512 -> 96

3. Parallelisation strategy

```
#pragma omp parallel for collapse(3) schedule(runtime) default(none) ...
for (n = 0; n < N; ++n)             /* batch          */
  for (c = 0; c < C; ++c)           /* channel        */
    for (oy = 0; oy < out_h; ++oy) /* output row: owned by one thread */
```

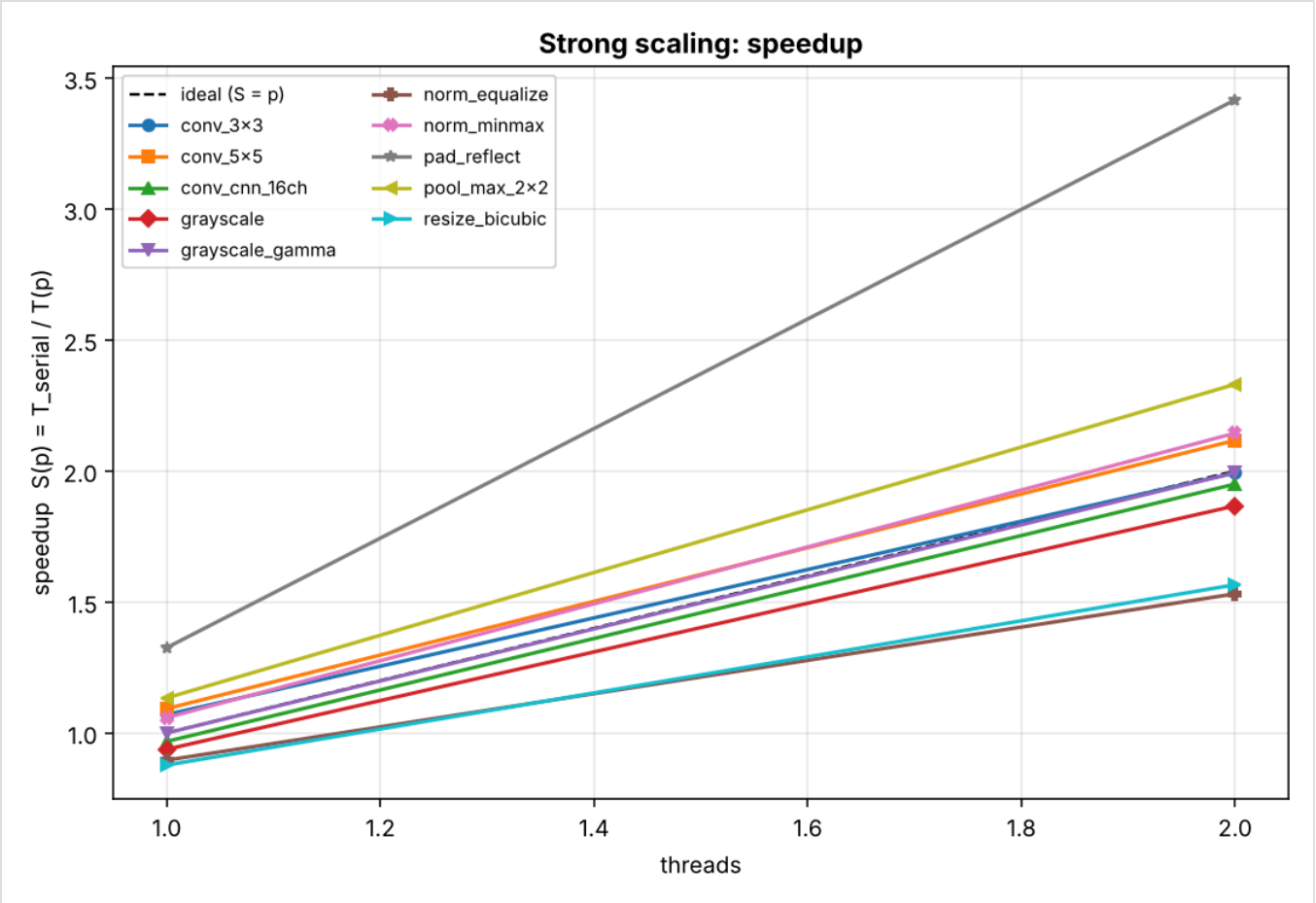
- **Row ownership.** Each iteration writes exactly one output row, so no two threads ever write the same element: no atomics, no locks, and only one barrier in the project (between the two resize passes).
- `collapse(3)` gives enough parallel work even for a single-channel image.

- **False sharing designed out.** Buffers are 64-byte aligned; per-thread histograms are padded to whole cache lines.
- **Reductions.** `reduction(min:/max:)` for min–max, two `reduction(+:)` sums in one pass for z-score, and private per-thread histograms (not atomics) for equalisation.
- **Left serial on purpose.** Resize weight tables are a few kilobytes; building them in parallel would cost more than it saves.

4. Results

Strong scaling on a 1024×1024×3 image, best of repeated runs after a warm-up, 2-core x86-64 Linux machine, GCC `-O3`, `schedule(static)` (data: `results/scaling.csv`):

Kernel	Bound by	Serial (ms)	2 threads (ms)	Speedup	Efficiency
`conv_3x3`	compute	27.95	14.03	1.99×	100 %
`conv_5x5`	compute	75.51	35.67	2.12×	106 %
`conv_cnn_16ch`	compute	90.38	46.35	1.95×	97 %
`grayscale_gamma`	compute	36.12	18.12	1.99×	100 %
`grayscale`	memory	0.60	0.32	1.87×	93 %
`norm_minmax`	memory	5.95	2.77	2.14×	107 %
`norm_equalize`	serial phase	6.57	4.29	1.53×	77 %
`resize_bicubic`	serial phase + barrier	16.91	10.80	1.57×	78 %
`pool_max_2x2`	memory	3.57	1.53	2.33×	117 %
`pad_reflect`	memory	2.10	0.61	3.42×	171 %



Arithmetic intensity predicts scalability. Compute-bound kernels (convolution, gamma-correct greyscale) reach about 2× on 2 cores. `norm_equalize` and `resize_bicubic` are limited by genuinely serial work — the histogram merge and LUT build, and the weight tables plus barrier — which the Karp–Flatt serial fraction confirms ($e \approx 0.31$ and 0.28).

Superlinear results, explained. `pad_reflect` and `pool_max_2x2` are so cheap (2–4 ms) that allocating and first-touching the output buffer is a large part of the measured time, and that work parallelises too. This is a measurement effect, not a property of the algorithms. An earlier 15× "speedup" was traced to a broken baseline (GCC's `if(0)` still runs the OpenMP loop machinery) and fixed by using directive-free serial code.

Chunk size is not a detail. With `schedule(dynamic)` OpenMP's default chunk is 1, so every pixel goes through the runtime's dispatcher:

Kernel	`static`	`dynamic`, chunk 1	`dynamic`, chunk 8192
`grayscale`	1.32×	**0.01×**	1.37×
`norm_minmax`	2.18×	**0.02×**	2.02×
`conv_5x5`	2.16×	2.07×	**1.07×**
`resize_bicubic`	1.68×	1.63×	**0.90×**

Per-pixel loops collapse at chunk 1; row loops (only ~1000 iterations) starve at chunk 8192. The chunk must match the loop granularity; for uniform work, `static` is the right default.

5. Correctness

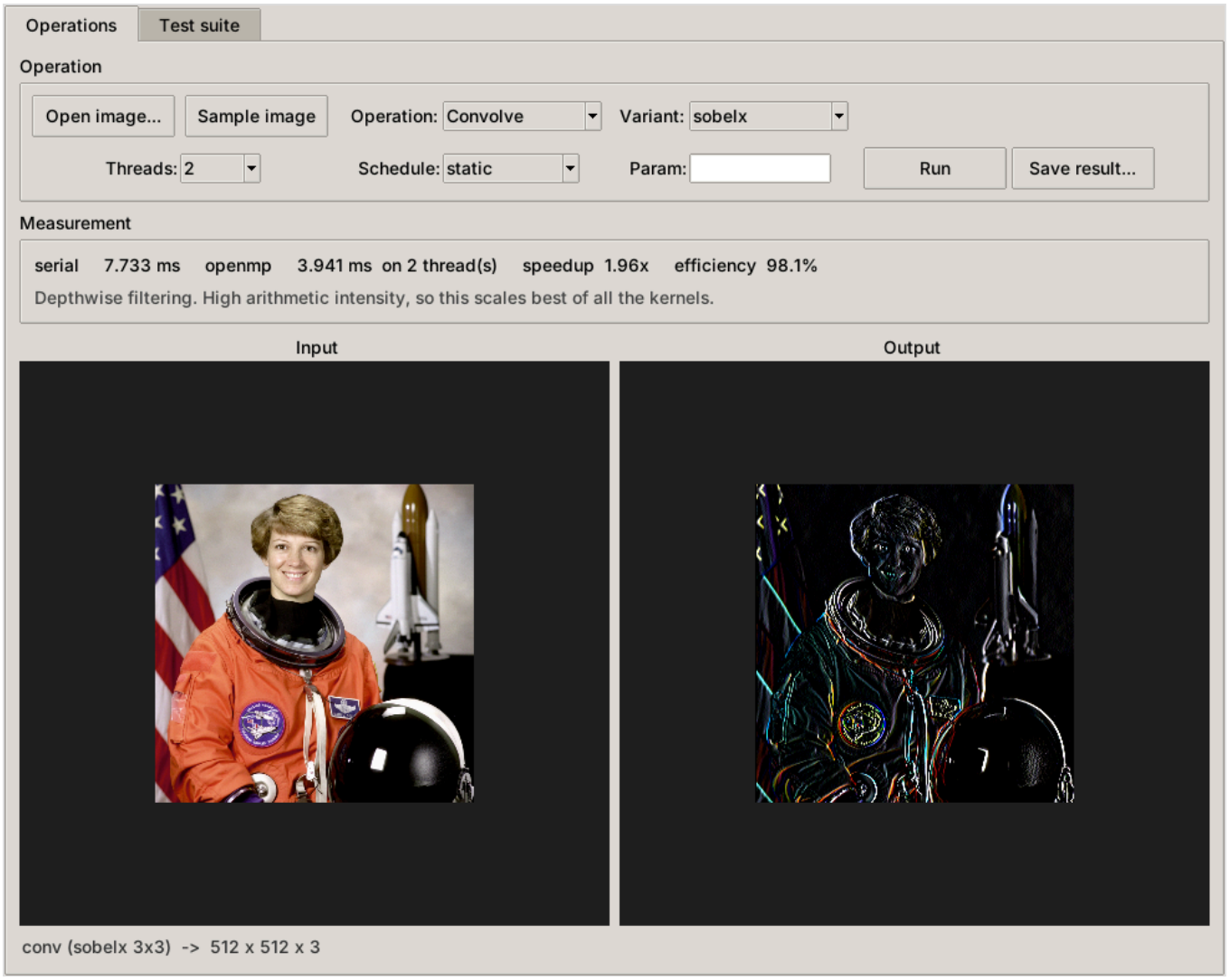
Check	Result
C test suite — 27 tests	**41,119 assertions pass**
OpenMP output = serial output	exact, every kernel
Output at 1, 2, 3, 4 threads and under static/dynamic/guided	bit-identical
ThreadSanitizer with Archer (OpenMP-aware)	**no data races**
AddressSanitizer + UndefinedBehaviorSanitizer	clean
Cross-check against an independent NumPy implementation	**20/20**, 0 grey-level difference except ±1 rounding in bilinear/bicubic/average
PNG/BMP/TGA round trips; PNG input = PPM input	lossless, identical
Continuous integration	Linux, macOS and Windows on every push

Two real bugs were found this way:

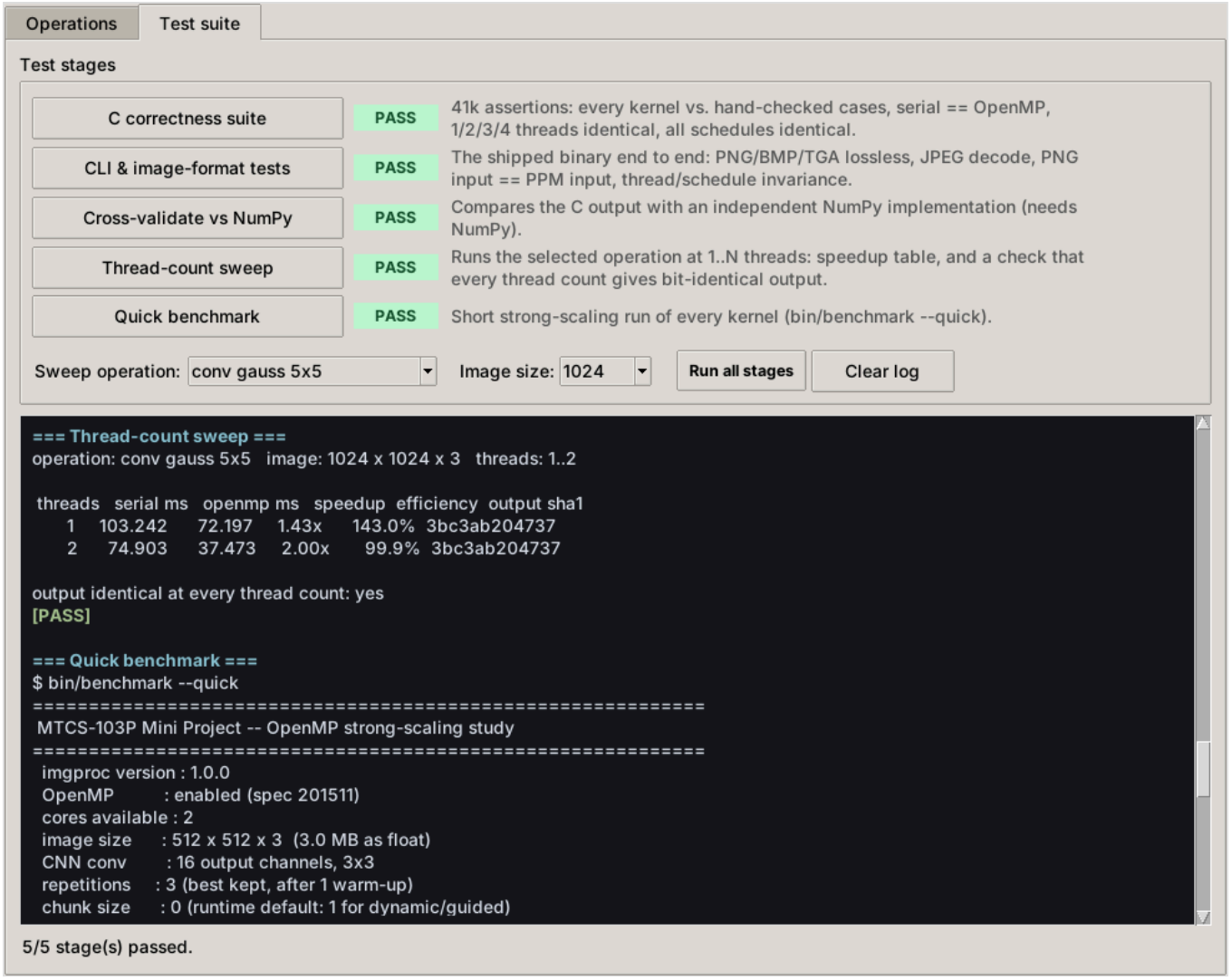
- **Resize (by cross-validation).** Nearest-neighbour downscaling disagreed with NumPy on 0.6 % of pixels: `floorf` in `float32` fell on the wrong side of an integer for ratios like 256/97. Fixed by computing sample positions in `double`.
- **Compiler bug (by the serial = OpenMP test).** On macOS, Apple clang 21 at `-O2`/`-O3` drops increments in a `schedule(runtime)` loop that updates a per-thread histogram — 27,648 pixels produced only 19,633 counts. GCC and `-O0`/`-O1` were correct. Writing the increment through a `volatile` pointer avoids the faulty optimisation at no measurable cost.

6. Graphical interface

`gui/app.py` (Python standard library only). **Operations:** load any image, choose operation, thread count and schedule, and compare serial and OpenMP time side by side with the result.



Test suite: one click per test stage — C suite, image-format tests, NumPy cross-check, thread-count sweep with output hashing, quick benchmark — each with a live PASS/FAIL badge.



7. How to run and conclusions

Run `RUN_LINUX_MAC.sh` / `RUN_WINDOWS.bat` (menu: build, verify, demo, benchmark, GUI), or `python3 run_project.py all`; `python3 verification.py` re-runs every check above. Requirements: a C compiler with OpenMP and Python 3.

1. **Arithmetic intensity decides scaling** — compute-bound kernels scale near ideally; memory-bound ones and those with serial phases do not.
2. **Row ownership removes synchronisation** — the decomposition mattered more than tuning directives.
3. **Chunk size must match loop granularity** — a wrong chunk cost up to 100×.
4. **Exact serial = parallel testing pays off** — it caught a compiler bug that reading the code never would.